

17. L1 & L2 Regularization (Lasso & Ridge)

★ Why Regularization?

⇒ When a machine learning model overfits, it learns the training data too well, including noise & unnecessary patterns.

This causes high training accuracy but poor test accuracy.

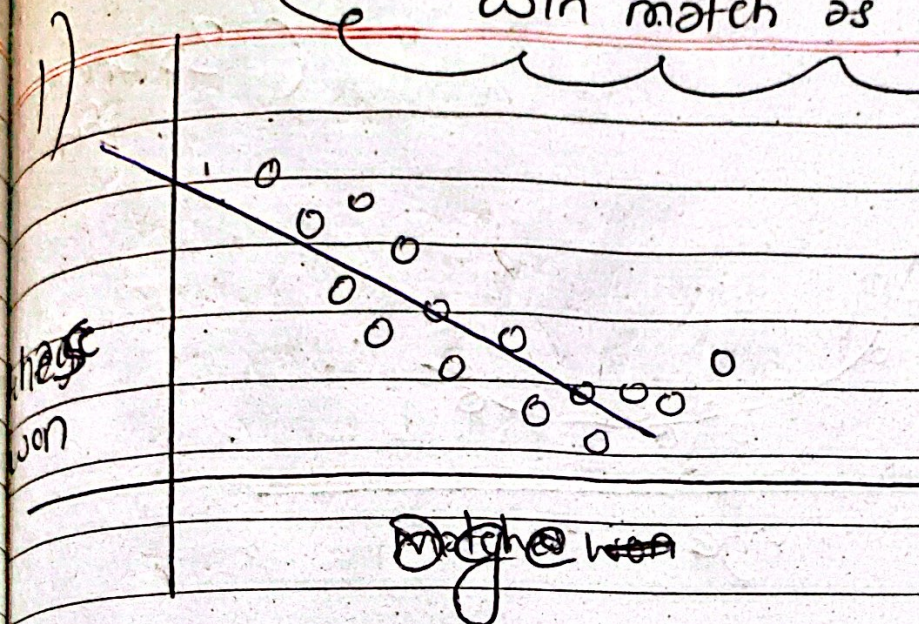
Regularization helps reduce overfitting by controlling the size of model parameters (θ values / coefficients).

★ Underfitting, Overfitting, Balanced fit

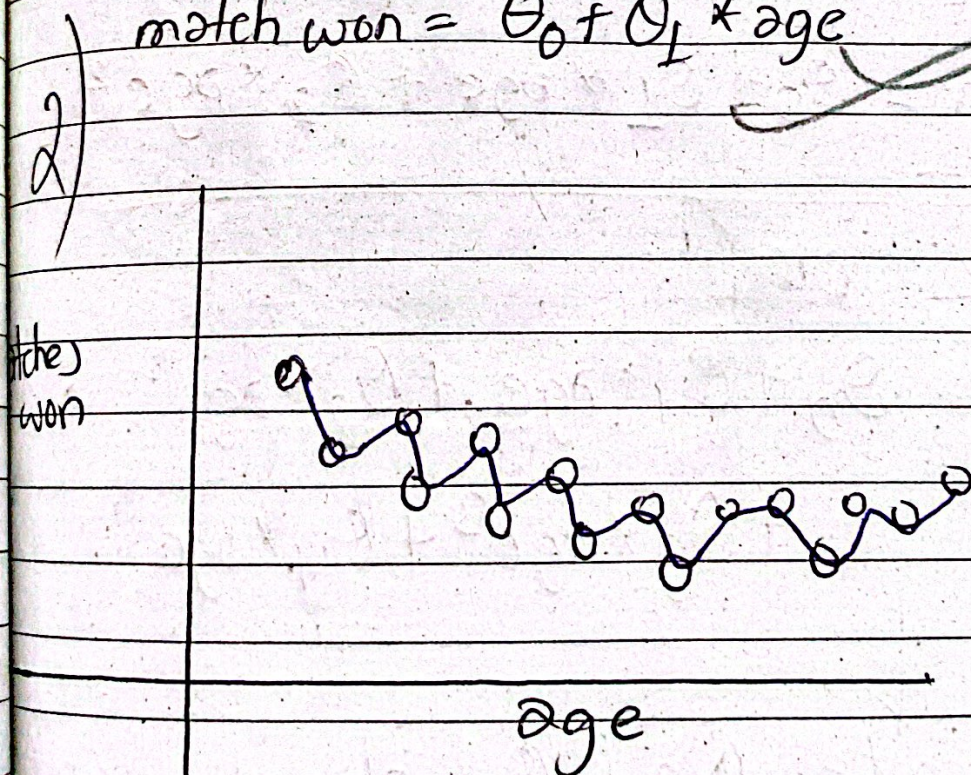
Underfitting

- Model is too simple
- cannot capture the pattern in data
- Example - Trying to fit a straight line to non linear data.

With increase in age of player, they can't win match as in their prime.



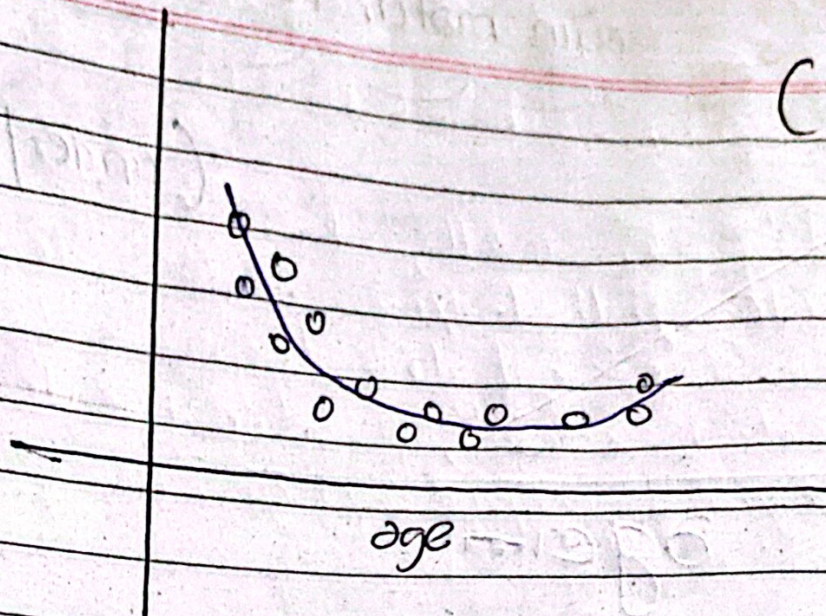
$$\text{match won} = \theta_0 + \theta_1 * \text{age}$$



$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2 + \theta_3 * \text{age}^3 + \theta_4 * \text{age}^4$$

3)

match won



(Balanced fit

Imp

$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2$$

What we have to do, overfit

2nd eqⁿ

$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2 + \theta_3 * \text{age}^3 + \theta_4 * \text{age}^4$$

Try to make θ_3 & θ_4 almost close to 0

$$\text{match won} = \theta_0 + \theta_1 * \text{age} + \theta_2 * \text{age}^2$$

Balance fit

Overfitting

- Model is too complex.
- fits even noise & random fluctuations.
- Works well on training data but poorly on unseen test data.
- Example: - Using a high-degree polynomial curve which zig-zags through all data points.

Balanced fit

- Model is neither too simple nor too complex.
- Generalizes well to new/unseen data.

How Regularization Reduces Overfitting

Suppose your model equation contains parameters:

$$h_{\theta}(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^5 + \theta_4 x_4^4$$

If the model overfits, usually some θ values become very large to force the curve through all points.

Regularization adds a penalty term to the loss function that discourages large θ values.

★ Mean Squared Error (Without regularization)

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$\hat{y}_i$ - Predicted

During training, the model tries to minimize this error.

★ L2 regularization (Ridge Regression)

Modified Loss function

$$Loss = MSE + \lambda \sum_{j=0}^n \theta_j^2$$

λ - hyperparameter (0 to ∞)
 θ_j - coefficients

Remember: MSE is 0.5

- The term θ_j^2 is the (L2 penalty).
- λ (lambda) is a hyperparameter controlling how strong the penalty is.

Remember: What we have to do

(large λ) $\Rightarrow$ 0 values. Thanks more simpler model

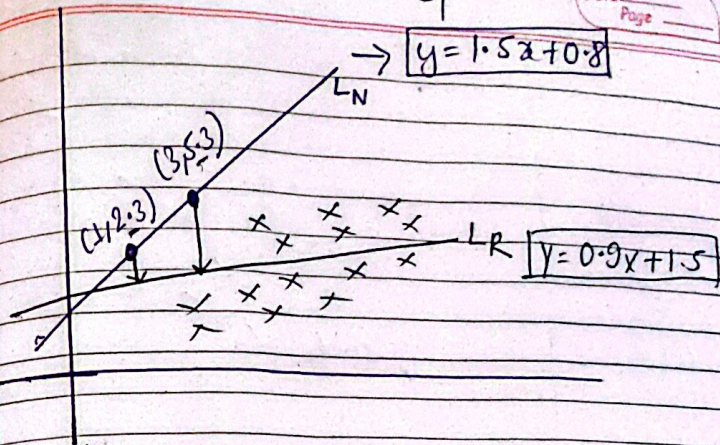
small $\lambda \rightarrow$ 0 values remain larger $\downarrow$ more complex model

$\lambda = 1$ (By default)

• train
x test

L2 (Ridge Regression)

eq



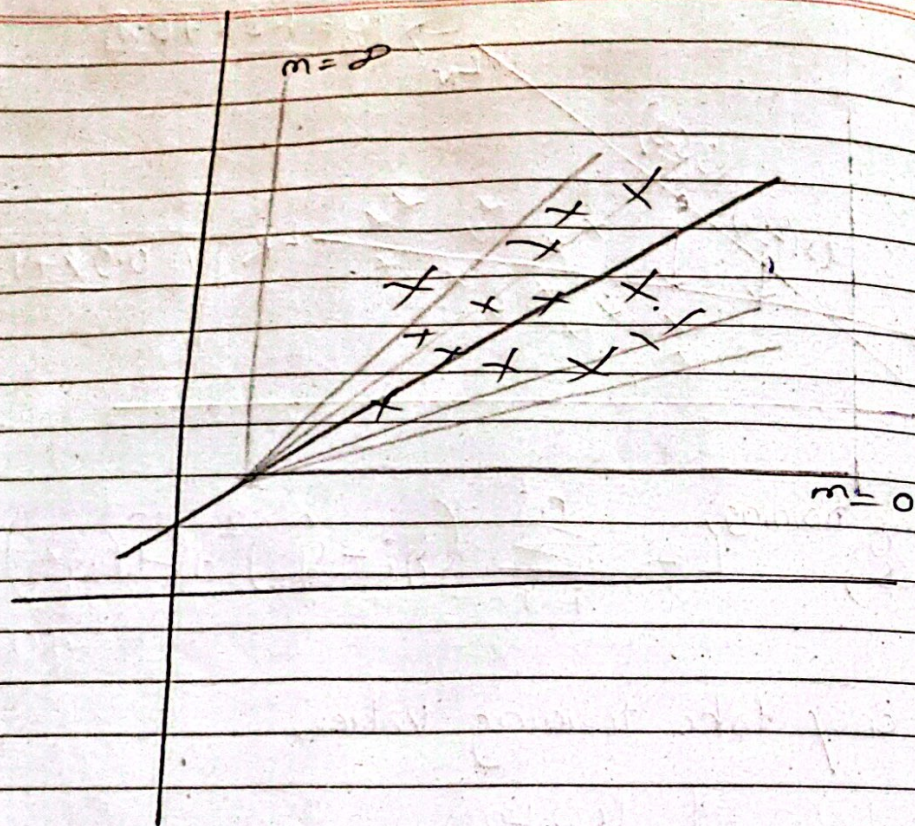
During training, (Loss) $L = \sum_{i=1}^n (y_i - \hat{y})^2 + \lambda (m^2)$

λ - hyperparameter.

So, we only take training values,

Loss L_N	Loss L_R
$\lambda = 1$	$\lambda = 1$
$(0)^2 + 1 \cdot (1.5)^2$ $= 2.25$	$L = (2.3 - (0.9 \cdot 2 + 1.5))^2 + (5.3 - (0.9 \cdot 3 + 1.5))^2 + 1 \cdot (0.9)^2$ $= 2.03$

So, it will choose L_R , even though L_R is doing a lot of mistakes in training than L_N .



$$y = ma + b$$

$m = \infty$, y depends heavily on x (overfitting)
 $y = mx$

$m = 0$, y does not depend on x (underfitting)
 $y = b$

Effects θ_3, θ_4

- prevents coefficients from growing too large
- does not make coefficients exactly zero, it only shrinks them.
- produces smoother models.

L1 Regularization (Lasso Regression)

Modified Loss function

$$\text{Loss} = \text{MSE} + \lambda \sum_{j=0}^n |\theta_j|$$

- The absolute value makes this L1 penalty
- L1 heavily penalizes large coefficients.

Special feature:-

- L1 can force some coefficients to become exactly zero, performing feature selection automatically.

Difference Between L1 & L2

feature	L1 (Lasso)	L2 (Ridge)
penalty coefficient		0
Usage	can become exactly zero feature selection	only shrink, never zero reducing overfitting
Solution	can be sparse	usually not sparse structured

Regularization Intuition

Regularization makes the model prefer simple equations, which generalize better.

If parameters (0) become too large, the penalty increases, so the model avoids them.

1. Simple Linear Regression

- Training accuracy: 68%
- Test accuracy: 14%
- clear overfitting

2. L1 Regularization - Lasso Regression

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=50)
model.fit(X_train, y_train)
```

Result:-

- Training accuracy $\approx 67\%$
- Test accuracy $\approx 66\%$

Major improvement from 14% $\rightarrow$ 66%.

shows overfitting is reduced.

3. L2 Regularization - Ridge Regression

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=50)
model.fit(X_train, y_train)
```

Result:-

- Training accuracy $\approx 67\%$
- Test accuracy $\approx 67\%$

Again large improvement.

Key Takeaways

1. Regularization improves model generalization.
2. L1 (Lasso) uses absolute value penalty & can remove features.
3. L2 (Ridge) uses square penalty & shrinks coefficients smoothly.
4. α controls the strength of regularization.
5. Without regularization, linear regression can easily overfit, especially with many features.